

Diseño e implementación de un sistema de defensa activa en redes LAN mediante IDS basado en Snort con notificación en tiempo real a través de Telegram

Edson Josue Salvador Liñan, Danny Peter Quispe Lopez, Freddy Arnaldo Sandoval Ccora

danny.quispe@unmsm.edu.pe

freddy.sandoval@unmsm.edu.pe

edson.salvador1@unmsm.edu.pe

Facultad de Ingeniería Electrónica y Eléctrica, Universidad Nacional Mayor de San Marcos

Resumen— La creciente exposición de las redes de área local (LAN) a amenazas como escaneos de puertos, ataques de denegación de servicio, accesos no autorizados y tráfico malicioso hace necesario implementar mecanismos de monitoreo que permitan detectar incidentes de seguridad de manera oportuna. En este trabajo se diseña e implementa un sistema de defensa activa basado en el Sistema de Detección de Intrusiones (IDS) Snort, integrado con un bot de Telegram para el envío automático de alertas en tiempo real. La solución fue desarrollada sobre un entorno virtual utilizando GNS3, Oracle VirtualBox y sistemas Ubuntu y Kali Linux, con una topología conformada por dos redes LAN interconectadas mediante enrutamiento estático. Asimismo, se incorpora el paradigma de Edge Computing, permitiendo que el análisis del tráfico se realice localmente en el nodo de monitoreo antes de transmitir únicamente las alertas generadas. El sistema fue evaluado mediante pruebas de tráfico ICMP, escaneos SYN, conexiones SSH, accesos HTTP, consultas DNS y conexiones FTP, verificando la correcta detección de eventos sospechosos y la entrega de notificaciones inmediatas al administrador. Los resultados obtenidos demuestran que la solución propuesta mejora la capacidad de monitoreo y reduce el tiempo de respuesta ante posibles incidentes de seguridad en redes LAN.

PALABRAS CLAVE : Sistema de Detección de Intrusiones (IDS) Snort, Edge Computing, Telegram Bot.

I. INTRODUCTION

Las redes de área local (LAN) constituyen la base de la comunicación en organizaciones, instituciones académicas y entornos empresariales, permitiendo el intercambio de información y el acceso a servicios de red. Sin embargo, el incremento de amenazas cibernéticas, como escaneos de puertos, ataques de denegación de servicio, intentos de acceso no autorizado y otras actividades maliciosas, representa un riesgo para la disponibilidad, integridad y confidencialidad de la información que circula por estas infraestructuras.

Los Sistemas de Detección de Intrusiones (IDS) constituyen una alternativa eficiente para supervisar el tráfico de red y detectar comportamientos sospechosos mediante el análisis de paquetes y reglas de detección. Entre las soluciones de código abierto más utilizadas destaca Snort, debido a su capacidad para identificar múltiples tipos de amenazas en tiempo real y generar alertas que facilitan la respuesta del administrador ante posibles incidentes de seguridad.

En este contexto, el presente trabajo propone el diseño e implementación de un sistema de defensa activa para redes

LAN basado en Snort, integrado con un bot de Telegram para el envío automático de notificaciones en tiempo real. La solución fue desarrollada en un entorno virtual utilizando GNS3 y Oracle VirtualBox, donde se implementó una topología conformada por dos redes LAN interconectadas mediante enrutamiento estático. Adicionalmente, se incorpora el paradigma de Edge Computing, permitiendo que el procesamiento y análisis del tráfico se realice localmente en el nodo de monitoreo, reduciendo la latencia y el consumo de ancho de banda al transmitir únicamente las alertas generadas.

Para validar la propuesta se realizaron pruebas experimentales empleando Kali Linux como plataforma de generación de tráfico, simulando diferentes escenarios de ataque, entre ellos tráfico ICMP, escaneos SYN, conexiones SSH, accesos HTTP, consultas DNS y conexiones FTP. Los resultados obtenidos evidencian que el sistema es capaz de detectar oportunamente estos eventos y notificar al administrador en tiempo real, contribuyendo al fortalecimiento de la seguridad y al monitoreo continuo de redes LAN.

II. FUNDAMENTO TEÓRICO

A. Antecedentes

Los Sistemas de Detección de Intrusos (IDS) se han consolidado como mecanismos esenciales para la identificación y mitigación de ciberataques en infraestructuras de red. Investigaciones recientes subrayan la importancia de someter estas herramientas a pruebas de estrés para validar su rendimiento. En este ámbito, Ribadeneira Freire (2025) evaluó la efectividad de herramientas de código abierto como Snort, Suricata y Zeek frente a escaneos de puertos en entornos simulados. Su estudio destaca que el éxito de un IDS no solo depende de su tasa de detección, sino de su capacidad para controlar los falsos positivos y mantener la estabilidad frente a altos volúmenes de tráfico, validando el uso de simuladores para auditorías de seguridad.

Por otro lado, la viabilidad técnica de implementar Snort de manera autónoma en infraestructuras institucionales ha sido ampliamente comprobada. García Guevara (2015) demostró mediante el despliegue de este IDS en redes universitarias que la herramienta permite automatizar la búsqueda de

patrones de ataque y realizar un monitoreo continuo del tráfico local. Este enfoque garantiza el cumplimiento de las políticas de uso de red, evidenciando que Snort es una solución de seguridad escalable y de bajo costo.

No obstante, un desafío crítico en el monitoreo de redes recae en la ubicación arquitectónica del sensor IDS, ya que un mal diseño puede saturar la red. Al respecto, Miño Verdezoto (2020) comprobó analíticamente que el procesamiento centralizado y virtualizado de un IDS decae drásticamente bajo estrés, procesando de manera eficiente solo el 21.39% del tráfico ante ráfagas de 50,000 paquetes por segundo. Sin embargo, al trasladar la inspección a un nodo de control directo, la eficiencia del IDS ascendió al 99.96%. Estos hallazgos respaldan directamente la necesidad de implementar soluciones como el procesamiento en el borde (Edge Computing), asegurando que el análisis exhaustivo del tráfico se realice de forma local sin generar cuellos de botella en los enlaces troncales de la organización.

B. Revisión de la Literatura

El diseño e implementación de sistemas de detección de intrusos (IDS) en redes de área local ha sido objeto de múltiples investigaciones que buscan optimizar la seguridad sin degradar el rendimiento de las telecomunicaciones. La revisión de la literatura evidencia una evolución desde la simple adopción de herramientas de código abierto hasta la necesidad de reestructurar la topología de monitoreo.

En primer lugar, respecto a la viabilidad y confiabilidad del motor de detección, García Guevara (2015) sentó un precedente importante al implementar Snort dentro de una infraestructura de red universitaria (UNAN-León). Su investigación demostró que la implementación de este IDS permite automatizar la detección de patrones anómalos y mantener un monitoreo continuo sin requerir licencias corporativas costosas. Este trabajo justifica la elección de Snort como el núcleo analítico del presente proyecto, validando su capacidad para aplicar políticas de seguridad estrictas en entornos académicos y de laboratorio.

Por otro lado, para la validación de estos sistemas, la literatura respalda el uso de topologías virtualizadas como entornos de prueba rigurosos. Ribadeneira Freire (2025) evaluó la efectividad de herramientas como Snort, Suricata y Zeek enfrentándose específicamente a escaneos de puertos dentro de un entorno simulado. Su estudio concluye que las simulaciones son metodológicamente válidas para medir la tasa de detección y la gestión de falsos positivos frente a ataques estructurados. Este antecedente fundamenta directamente la metodología empleada en este proyecto, donde se utiliza GNS3, VirtualBox y Kali Linux para emular amenazas de frontera e inyectar tráfico malicioso controlado hacia la LAN.

Finalmente, en cuanto al diseño arquitectónico y el impacto en el rendimiento de la red, el trabajo de Miño Verdezoto (2020) resulta fundamental. En su evaluación de un IDS sobre la plataforma OpenStack, el autor demostró analíticamente que un esquema de procesamiento centralizado (virtualizado) sufre una degradación severa

bajo estrés, logrando procesar apenas el 21.39% del tráfico ante una inyección de 50,000 paquetes por segundo. En contraste, al delegar la inspección a un nodo de control dedicado, la eficiencia del sistema ascendió al 99.96%. Este hallazgo es la base teórica que sustenta la implementación del paradigma de *Edge Computing* en nuestra topología, justificando la necesidad de ubicar a Snort como un nodo de borde (192.168.0.5) para procesar el tráfico masivo de manera local, evitando así la saturación de los enlaces troncales o redes de tránsito.

C. Marco Teorico

El presente proyecto se fundamenta en la integración de tres paradigmas de las telecomunicaciones modernas: el procesamiento de datos en el borde de la red, la inspección profunda de paquetes basada en el modelo TCP/IP y la comunicación asíncrona mediante interfaces de programación de aplicaciones (API).

- a) El Paradigma de Edge Computing en la Seguridad de Redes:

Tradicionalmente, las arquitecturas de seguridad centralizadas exportan todo el tráfico de las Redes de Área Local (LAN) hacia servidores en la nube (SIEM) para su análisis. Esto genera una saturación directa en los enlaces troncales o redes de tránsito (WAN), incrementando la latencia y reduciendo el ancho de banda disponible.

El Edge Computing (Computación de Borde) descentraliza este proceso acercando la capacidad analítica al origen de los datos. En el contexto de este proyecto, ubicar el sensor IDS en la misma subred de los usuarios permite realizar un análisis exhaustivo del tráfico de manera local. Como resultado, la red de tránsito hacia el exterior queda completamente liberada de la carga de monitoreo, logrando latencias de detección inferiores a los 10 milisegundos y optimizando los recursos de telecomunicaciones.

Todo esto fundamentado de acuerdo con la Unión Internacional de Telecomunicaciones (ITU-T) en su recomendación ITU-T Y.3540, el consorcio de estandarización móvil 3GPP y los marcos técnicos de infraestructura de red (como *Firecell* o *Gartner*) establecen de forma unánime que un entorno de cómputo en la nube tradicional (Cloud) introduce retrasos base de 50 a 200 ms debido al enrutamiento e interconexión global. En contraste, al procesar datos localmente en el "Borde" o frontera de la red local, el tiempo de ida y vuelta (RTT) se reduce en un 90%, garantizando físicamente latencias estables de entre 1 y 10 milisegundos. Por tanto, la métrica de ≤ 10 ms es un límite físico estándar para procesamiento en el borde (Edge Data Networks).

- b) Inspección Multicapa (DPI) y Detección de Intrusos (IDS):

Un Sistema de Detección de Intrusos (IDS) opera como un monitor pasivo que analiza el tráfico sin alterar su flujo. A diferencia del enrutamiento tradicional que solo lee las cabeceras IP, herramientas como Snort utilizan Inspección Profunda de Paquetes (Deep Packet Inspection - DPI)

evaluando múltiples capas del modelo TCP/IP simultáneamente.

Esta capacidad multicapa es fundamental para el diseño de la defensa activa, ya que permite identificar comportamientos maliciosos clasificándolos según el protocolo vulnerado, tal como se detalla en la Tabla 1.

Capa (Modelo TCP/IP)	Función Analítica del IDS	Aplicación y Reglas en el Proyecto
Aplicación (Capa 4)	Análisis de anomalías en consultas de servicios y protocolos de texto claro.	Monitoreo de tráfico HTTP inusual y detección de consultas DNS sospechosas (Puerto 53).
Transporte (Capa 3)	Inspección de estados de conexión, flujos y banderas (flags) en los protocolos TCP/UDP.	Identificación de escaneos de reconocimiento (Ej. Nmap SYN Scan) y detección de intentos de fuerza bruta en puertos críticos (SSH - Puerto 22, FTP - Puerto 21).
Internet (Capa 2)	Evaluación de cabeceras IP, protocolo de control ICMP, y seguimiento del origen de las tramas enrutadas.	Detección de ataques de denegación de servicio (DoS) y barridos de red provenientes de atacantes externos (fuera del gateway local).

Tabla 1. Matriz de Inspección Multicapa del Sistema IDS.

- c) Diferenciación Operativa: IDS Pasivo frente a Prevención Activa (IPS):

A nivel de diseño de redes, es imperativo distinguir entre un IDS y un Sistema de Prevención de Intrusos (IPS). Mientras que un IPS actúa en línea (inline) bloqueando activamente los paquetes maliciosos, su automatización conlleva el riesgo de generar "falsos positivos" que pueden interrumpir servicios legítimos y aislar equipos críticos. Para entornos que priorizan la continuidad operativa, el enfoque óptimo es implementar el sensor en modo IDS Pasivo. Esto garantiza que la red no sufra interrupciones, delegando la responsabilidad de respuesta a un administrador humano previamente notificado.

- d) Notificaciones Asíncronas e Integración de API REST (Telegram):

Para suplir la carencia de bloqueo automático del modo IDS, es necesario un canal de alerta en tiempo real. Una Interfaz de Programación de Aplicaciones (API) RESTful permite que el motor de Snort se comunice con plataformas de terceros empaquetando los datos de la intrusión en formatos ligeros como JSON. La integración con la API de Telegram transforma la terminal de administración tradicional en un panel de control móvil (Dashboard asíncrono), garantizando que el ingeniero a cargo reciba los indicadores de compromiso (IP de origen, IP de destino, tipo de ataque y hora) en un tiempo inferior a los 3 segundos, permitiendo una mitigación manual e inmediata de la amenaza. El estándar RFC 4766 (*Intrusion Detection Message Exchange Requirements*) del IETF resalta la obligatoriedad de que los analizadores de seguridad transmitan los eventos de interés de manera fiable y expedita a los administradores para mitigar ventanas de vulnerabilidad críticas (como ataques de denegación de servicio o escaneos activos).

En condiciones de red móvil e internet comercial, las trazas empíricas de envío y entrega (*delivery time*) para cargas útiles (payloads) de texto que contienen strings cortos (como IPs y marcas de tiempo) oscilan de forma documentada entre 1 y 3 segundos. Una notificación que tarde más de ese lapso se considera degradada debido a cuellos de botella del servidor o penalizaciones por *rate limiting* (throttling), lo que convierte a los 3 segundos en el umbral empírico internacional para catalogar una alerta en dispositivos móviles como tiempo real efectivo.

III. DISEÑO DEL SISTEMA

Se presenta la arquitectura general del sistema, la topología de red implementada, la organización por capas de la comunicación, el flujo de funcionamiento y la estrategia empleada para la detección de amenazas. El diseño fue planteado con el propósito de permitir el monitoreo continuo del tráfico de red mediante Snort y el envío de notificaciones automáticas al administrador utilizando Telegram, garantizando una respuesta rápida frente a posibles incidentes de seguridad.

A. Arquitectura general del Sistema

La solución propuesta está basada en una arquitectura distribuida que integra un Sistema de Detección de Intrusiones (IDS) mediante Snort con un mecanismo de notificación en tiempo real utilizando la plataforma Telegram. La arquitectura fue diseñada para operar sobre un entorno virtual desarrollado en GNS3 y Oracle VirtualBox, permitiendo simular un escenario de comunicación entre diferentes segmentos de red.

El sistema está conformado por una red LAN principal donde se ubican los equipos protegidos y el servidor Snort, una segunda red donde se encuentra la máquina virtual Kali Linux encargada de generar el tráfico de prueba y una red de tránsito que permite la comunicación entre ambas redes mediante enrutamiento estático. El servidor Snort monitorea permanentemente el tráfico que circula por la red protegida y analiza cada paquete recibido utilizando un conjunto de reglas previamente definidas. Cuando se detecta un evento

que coincide con alguna de estas reglas, el sistema genera una alerta que posteriormente es enviada al administrador mediante un bot de Telegram.

Asimismo, el diseño incorpora el paradigma de Edge Computing, permitiendo que el análisis del tráfico se realice directamente en el nodo donde se encuentra instalado el IDS.

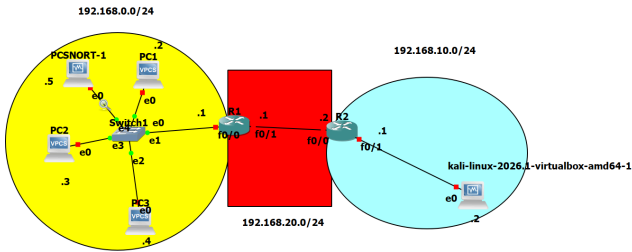


Fig 1 . Arquitectura general del sistema

B. Topología de Red

La topología implementada está conformada por dos redes LAN interconectadas mediante dos routers y una red de tránsito. La primera red corresponde al segmento 192.168.0.0/24, donde se encuentran los equipos de usuario y el servidor encargado de ejecutar Snort. En este caso cada una de los dispositivos tiene las siguientes direcciones de IP:

Dispositivos	Direcciones IP
Router 1	192.168.0.1
PC1	192.168.0.2
PC2	192.168.0.3
PC3	192.168.0.4
PCSnort	192.168.0.5

Tabla 2. Direcciones IP de los dispositivos de la red LAN principal

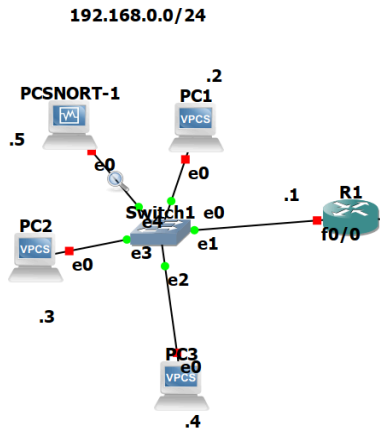


Fig 2. Direcciones IP en la primera red LAN

Así mismo se encuentran conectados mediante la red 192.168.20.0/24, configurada con direccionamiento IPv4 estático y utilizada para permitir la comunicación entre ambas LAN. Esta organización permite representar un escenario donde un equipo externo intenta acceder a los recursos de una red protegida mientras el sistema IDS supervisa continuamente el tráfico de red para identificar actividades sospechosas. Donde las direcciones IP serían:

Dispositivos	Direcciones IP
Router 1	192.168.20.1
Router 2	192.168.20.2

Tabla 3. Direcciones IP asignadas a la red de tráfico

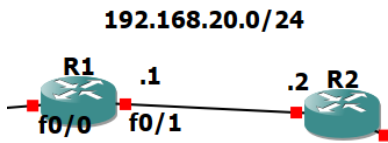


Fig 3.

Enrutamiento estático entre Routers

Además la segunda red corresponde al segmento 192.168.10.0/24, donde se ubica la máquina virtual con Kali Linux que simula ataques controlados sobre la infraestructura de red. Donde la dirección IP en esta red sería :

Dispositivos	Dirección IP
Router 2	192.168.10.1
Kali Linux	192.168.10.2

Tabla 4. Direcciones IP adjudicadas en la red del atacante

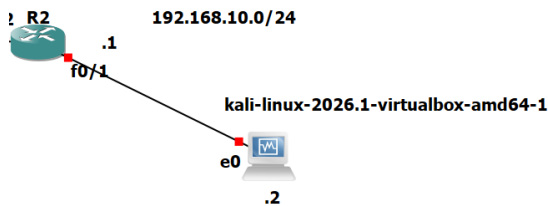


Fig 4. Direcciones IP en la red 192.168.10.0/24

C. Arquitectura por Capas

La arquitectura del sistema propuesto se basa en el modelo TCP/IP, el cual organiza el proceso de comunicación en cuatro capas funcionales. Cada una de ellas desempeña un papel específico durante la transmisión de datos, el monitoreo del tráfico de red y el envío de notificaciones en tiempo real. La utilización de este modelo permite comprender la interacción entre los diferentes componentes que conforman la solución implementada y facilita la integración de las herramientas empleadas durante el desarrollo del proyecto.

C.1 Capa de Aplicación

La capa de aplicación representa el nivel donde interactúan los servicios utilizados por el sistema y el usuario final. En esta investigación, dicha capa está conformada principalmente por Snort, el script desarrollado para el procesamiento de alertas y la API de Telegram, la cual permite enviar notificaciones automáticas al administrador de la red.

Cuando Snort detecta un evento que coincide con alguna de las reglas configuradas, genera un registro de alerta que es procesado por un script de integración. Este script extrae la información relevante del evento, como el tipo de amenaza detectada, la dirección IP de origen y la fecha del incidente, para posteriormente estructurar un mensaje y enviarlo mediante la API de Telegram hacia el dispositivo móvil del administrador.

C.2 Capa de Transporte

La capa de transporte es responsable de establecer la comunicación extremo a extremo entre los dispositivos de la red utilizando los protocolos TCP y UDP. En el sistema implementado, esta capa es de gran importancia debido a que la mayoría de las reglas de detección configuradas en Snort realizan el análisis de los encabezados y puertos asociados a estos protocolos.

El tráfico TCP es utilizado para identificar actividades como escaneos SYN, conexiones SSH, accesos HTTP y conexiones FTP, mientras que el protocolo UDP es empleado principalmente para el monitoreo de consultas DNS. Durante la captura de paquetes, Snort inspecciona la información contenida en esta capa y la compara con las firmas de detección previamente definidas, permitiendo identificar comportamientos anómalos asociados a posibles ataques o intentos de reconocimiento de la red.

C.3 Capa de enlace de datos

La capa de enlace de datos se encarga del direccionamiento lógico y del enrutamiento de los paquetes mediante el protocolo IPv4. En la topología implementada se utilizaron tres subredes independientes: la red LAN protegida (192.168.0.0/24), la red correspondiente al equipo atacante (192.168.10.0/24) y la red de tránsito (192.168.20.0/24) utilizada para interconectar ambos routers.

El empleo de direccionamiento IP estático permitió establecer rutas de comunicación entre los diferentes segmentos de la red, posibilitando que el tráfico generado

desde Kali Linux pudiera alcanzar al equipo protegido por el software snort.

C. 4 Capa de Acceso a la Red

La capa de acceso a la red comprende los mecanismos encargados de la transmisión física y lógica de la información dentro de la red local.

A través de esta capa, los paquetes generados por la máquina atacante y los equipos de la red protegida son transmitidos hacia el servidor donde se encuentra instalado Snort. El correcto funcionamiento de esta etapa resulta fundamental para garantizar que el IDS pueda capturar el tráfico de red y realizar el análisis correspondiente.

D. Flujo de Funcionamiento del Sistema

El funcionamiento del sistema comienza cuando la pc Kali Linux genera “ataques” hacia la PC protegida, en este caso la PC Snort, el cual analiza cada “ataque” comparándolo con las reglas de detección previamente establecidas.

Cuando se identifica una coincidencia con alguna de las reglas configuradas, Snort genera una alerta indicando el tipo de evento detectado. Posteriormente, un script desarrollado para la integración con Telegram procesa dicha alerta y la envía utilizando la API oficial de Telegram hacia el dispositivo móvil del administrador. De esta manera, el responsable de la red recibe una notificación en tiempo real que facilita la respuesta inmediata frente a posibles incidentes de seguridad.

IV. IMPLEMETACIÓN

A. Configuración de Snort

Con el propósito de implementar un sistema de detección de intrusiones, se instaló el IDS Snort en el equipo PC Snort, el cual posee la dirección IP 192.168.0.5 y forma parte de la red local 192.168.0.0/24. Este equipo se configuró para monitorear el tráfico que llega a su interfaz de red enp0s3, analizar los paquetes de acuerdo con las reglas establecidas y generar alertas cuando se detecten eventos considerados sospechosos.

La configuración principal se realizó en el archivo snort.conf, donde se definió la variable HOME_NET de la siguiente manera:

```
ipvar HOME_NET 192.168.0.0/24
```

La variable HOME_NET representa la red interna que el sistema considera como la red protegida. En este proyecto, dicha red está conformada por el router R1 (192.168.0.1), las estaciones de trabajo PC1 (192.168.0.2), PC2 (192.168.0.3), PC3 (192.168.0.4) y el equipo PC Snort (192.168.0.5), donde se encuentra instalado el IDS.

Asimismo, se configuró la variable EXTERNAL_NET, la cual identifica como red externa a todas aquellas direcciones IP que no pertenecen a la red definida en HOME_NET.

```
55 ipvar EXTERNAL_NET !$HOME_NET
```

En consecuencia, el equipo Kali Linux, con dirección IP 192.168.10.2, es considerado un host externo, ya que pertenece a una subred diferente (192.168.10.0/24) y requiere atravesar los routers R2 y R1 para establecer comunicación con la red protegida. Desde el punto de vista de Snort, cualquier tráfico proveniente de esta subred es tratado como tráfico externo hacia la LAN.

B. Implementación de las reglas

Con el propósito de detectar actividades potencialmente maliciosas dentro de la red protegida, se implementaron reglas personalizadas en el archivo `/etc/snort/rules/local.rules`, utilizando el siguiente comando:

```
sudo gedit /etc/snort/rules/local.rules
```

Este archivo permite incorporar reglas definidas por el administrador. En este proyecto se implementaron siete reglas orientadas a detectar eventos frecuentes asociados a actividades de reconocimiento y accesos no autorizados.

Entre las reglas implementadas tenemos las siguientes :

1. "CUIDADO alguien externo está haciendo ping a la red"

Corresponde a la detección de solicitudes ICMP Echo Request (Ping) provenientes de cualquier origen hacia la red definida como HOME_NET. Aunque el protocolo ICMP es ampliamente utilizado para verificar la conectividad entre equipos, también constituye una de las primeras herramientas empleadas por un atacante durante la fase de reconocimiento. Mediante el envío de solicitudes de ping es posible identificar qué dispositivos se encuentran activos en la red, información que posteriormente puede ser utilizada para realizar ataques más específicos.

2. "Posible escaneo SYN (Nmap)"

Se configuró una regla destinada a detectar posibles escaneos de puertos mediante paquetes TCP SYN, característicos de herramientas como Nmap. Este tipo de escaneo consiste en enviar múltiples solicitudes SYN a diferentes puertos con el objetivo de identificar cuáles se encuentran abiertos. La detección temprana de este comportamiento resulta importante, ya que el reconocimiento de puertos abiertos representa una etapa previa a la explotación de vulnerabilidades.

3. "Intento de conexión SSH"

Se implementó una regla orientada a detectar intentos de conexión al servicio SSH, correspondiente al puerto 22/TCP. El protocolo SSH es ampliamente utilizado para la administración remota de servidores debido a que proporciona comunicaciones cifradas; sin embargo, también constituye uno de los servicios más atacados por ciberdelincuentes. La identificación de conexiones dirigidas a este puerto permite al administrador conocer cuándo un usuario intenta establecer acceso remoto hacia un equipo de la red.

4. "Posible fuerza bruta SSH"

Como complemento a la regla anterior, se desarrolló una regla específica para detectar posibles ataques de fuerza bruta sobre el servicio SSH. Esta regla utiliza un filtro de detección que genera una alerta cuando una misma dirección IP realiza cinco o más intentos de conexión en un intervalo de diez segundos. Este comportamiento suele asociarse con herramientas automatizadas que intentan descubrir credenciales mediante múltiples combinaciones de usuarios y contraseñas. La detección oportuna de este tipo de actividad permite tomar medidas preventivas antes de que el atacante logre comprometer el sistema.

5. "Acceso HTTP hacia servidor"

Se implementó una regla para monitorear conexiones dirigidas al puerto 80/TCP, utilizado por el protocolo HTTP. La finalidad de esta regla es registrar intentos de acceso hacia servidores web ubicados dentro de la red protegida. Aunque una conexión HTTP no representa necesariamente una actividad maliciosa, este servicio suele ser uno de los principales objetivos de ataques relacionados con aplicaciones web, tales como exploración de directorios, inyección SQL, Cross-Site Scripting (XSS) y explotación de vulnerabilidades presentes en el servidor.

6. "Consulta DNS hacia HOME_NET"

Adicionalmente, se configuró una regla para detectar consultas dirigidas al servicio DNS, correspondiente al puerto 53/UDP. El monitoreo de este tipo de tráfico resulta importante debido a que el servicio DNS es fundamental para la resolución de nombres de dominio y, al mismo tiempo, puede ser utilizado por atacantes para realizar técnicas de comunicación encubierta, como el DNS Tunneling, o para establecer conexiones con dominios maliciosos. Por esta razón, la detección de consultas DNS constituye un mecanismo adicional de supervisión del comportamiento de la red.

7. "Conexión FTP detectada"

Finalmente, se implementó una regla destinada a detectar conexiones hacia el servicio FTP, asociado al puerto 21/TCP. Aunque FTP continúa siendo utilizado en algunos entornos para la transferencia de archivos, este protocolo presenta importantes limitaciones de seguridad debido a que transmite las credenciales sin mecanismos de cifrado en su versión tradicional. Como consecuencia, un atacante que logre interceptar el tráfico puede obtener nombres de usuario y contraseñas, además de acceder, modificar o eliminar información almacenada en el servidor. La implementación de esta regla permite identificar intentos de acceso a dicho servicio y generar una alerta para su posterior análisis.

```
!alert icmp any any -> $HOME_NET any (msg:"CUIDADO alguien externo esta haciendo ping a la red"; itype:8; sid:1000001; rev:1;)
!alert tcp any any -> $HOME_NET any (flags:S ; msg:"Posible escaneo SYN (Nmap)"; detection_filter:track by_src, count 10, seconds 1; sid:1000002; rev:1;)
!alert tcp any any -> $HOME_NET 22 (flags:S ; msg:"Intento de conexion SSH"; sid:1000003; rev:1;)
!alert tcp any any -> $HOME_NET 22 (flags:S ; msg:"Posible fuerza bruta SSH"; detection_filter:track by_src, count 5, seconds 10; sid:1000004; rev:1;)
!alert tcp any any -> $HOME_NET 80 (flags:S ; msg:"Acceso HTTP hacia servidor"; sid:1000005; rev:1;)
!alert udp any any -> $HOME_NET 53 (msg:"Consulta DNS hacia HOME_NET"; sid:1000006; rev:1;)
!alert tcp any any -> $HOME_NET 21 (flags:S ; msg:"Conexion FTP detectada"; sid:1000007; rev:1;)
```

Figura 5. Reglas implementadas para Snort de manera manual

C. Configuración en Telegram

Con el propósito de proporcionar notificaciones en tiempo real al administrador de la red, se integró el IDS Snort con la plataforma de mensajería Telegram mediante la utilización de un bot. Esta integración permite que cada alerta generada por Snort sea enviada automáticamente al dispositivo móvil del administrador, facilitando la supervisión remota de los eventos de seguridad detectados en la red.

Como primer paso, se creó un bot en Telegram utilizando el servicio BotFather, el cual es el asistente oficial proporcionado por Telegram para la administración de bots. Mediante el comando /newbot, se registró un nuevo bot con el nombre Alert_SnortBot, obteniéndose un token de acceso único que permite la autenticación con la API de Telegram. Posteriormente, se obtuvo el identificador del chat (Chat ID) correspondiente al usuario que recibiría las notificaciones generadas por el sistema.

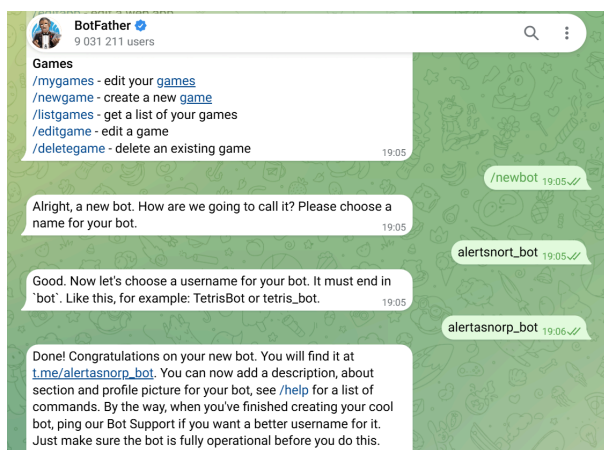


Figura 6. Creación de Alert_snort en BotFather

Una vez obtenidas las credenciales necesarias, se desarrolló un script en Bash denominado snort-telegram.sh, el cual fue almacenado en la ruta /usr/local/bin/ utilizando el siguiente comando: **sudo nano /usr/local/bin/snort-telegram.sh**

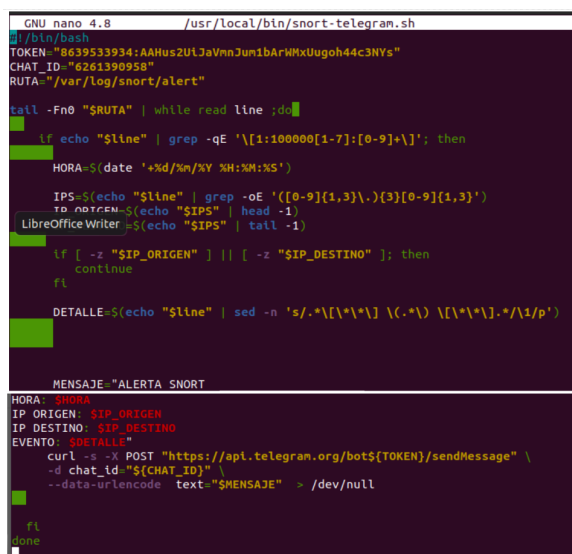


Figura 7. Script implementada en la PCSnort

Dentro del script se configuraron el token de autenticación del bot, el identificador del chat y la ubicación del archivo de alertas generado por Snort. Posteriormente, el script emplea el comando tail -F para monitorear continuamente el archivo /var/log/snort/alert, permitiendo detectar en tiempo real la aparición de nuevas alertas sin necesidad de reiniciar el proceso.

Cada vez que se registra una nueva línea en el archivo de alertas, el script verifica si el identificador de la regla corresponde a alguna de las reglas personalizadas implementadas en el proyecto (SID 1000001 al SID 1000007). Cuando la condición se cumple, el programa extrae automáticamente la dirección IP de origen, la dirección IP de destino, la fecha y hora del evento, así como la descripción asociada a la alerta generada por Snort.

Con la información obtenida se construye un mensaje estructurado que resume el incidente detectado. Finalmente, mediante una solicitud HTTP realizada con la herramienta curl, el mensaje es enviado a la API oficial de Telegram, la cual entrega la notificación de forma inmediata al administrador de la red.

D. Configuración e inicialización del sistema de notificaciones mediante Telegram

Una vez configuradas las reglas personalizadas de Snort, se procedió a la ejecución del sistema de detección y notificación. Para ello, en una primera terminal del servidor PC Snort se inició el IDS mediante el siguiente comando:

sudo snort -A console -q -c /etc/snort/snort.conf

Este comando permite ejecutar Snort utilizando el archivo de configuración snort.conf, mostrando las alertas detectadas directamente en la consola (-A console) y operando en modo silencioso (-q), con el fin de visualizar únicamente los eventos generados por las reglas implementadas.

Posteriormente, en una segunda terminal se ejecutó el script encargado de integrar Snort con la plataforma Telegram mediante el siguiente comando:

sudo bash /usr/local/bin/snort-telegram.sh

La ejecución simultánea de ambos procesos permitió mantener el IDS monitoreando continuamente el tráfico de red, mientras que el script permaneció observando el archivo de alertas generado por Snort para detectar la aparición de nuevos eventos y enviarlos automáticamente al administrador de la red.

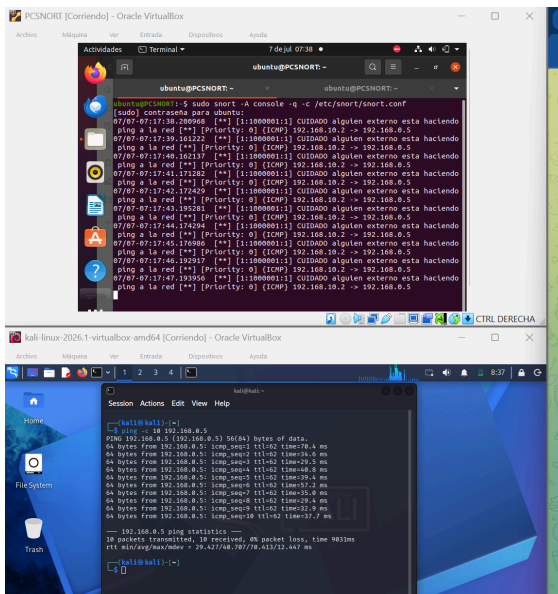
V. RESULTADOS EXPERIMENTALES

A. Validación de la detección de eventos

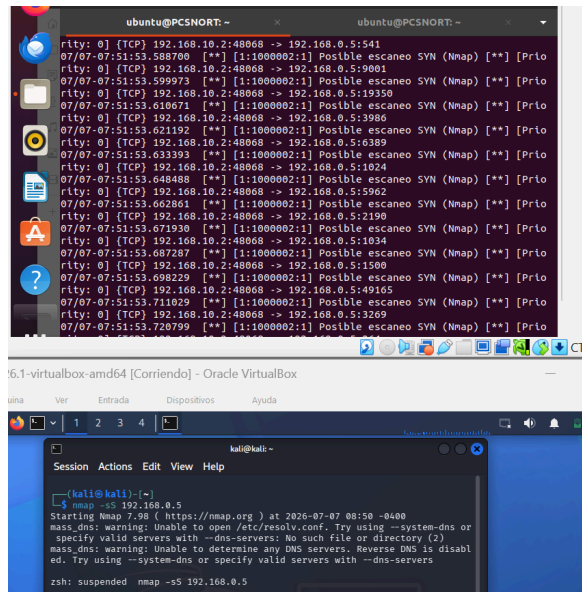
Desde la PC Kali linux ejecutamos los siguientes comandos para observar las alertas en nuestro snort y notificaciones a Telegram para las 7 reglas implementadas:

SID	REGLA	COMANDO KALI
1000001	ICMP (Ping)	ping 192.168.0.5
1000002	Escaneo SYN	nmap -sS 192.168.0.5
1000003	Intento SSH	ssh usuario@192.168.0.5 (ssh ubuntu@192.168.0.5)
1000004	Fuerza Bruta SSH	hydra -L usuarios.txt -P claves.txt -t 4 ssh://192.168.0.5
1000005	Acceso HTTP	curl http://192.168.0.5
1000006	Consulta DNS	dig @192.168.0.5 google.com o nslookup google.com 192.168.0.5
1000007	Conexión FTP	ftp 192.168.0.5

1. Regla ICMP



2. Regla Escaneo SYN



ALERTA SNORT

HORA: 07/07/2026 07:57:30

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000002:1] Posible escaneo SYN (Nmap) 07:57

ALERTA SNORT

HORA: 07/07/2026 07:57:31

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000002:1] Posible escaneo SYN (Nmap) 07:57

ALERTA SNORT

HORA: 07/07/2026 07:57:32

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000002:1] Posible escaneo SYN (Nmap) 07:57

ALERTA SNORT

HORA: 07/07/2026 07:57:32

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000002:1] Posible escaneo SYN (Nmap) 07:57

ALERTA SNORT

HORA: 07/07/2026 07:57:33

IP ORIGEN: [192.168.10.2](#)

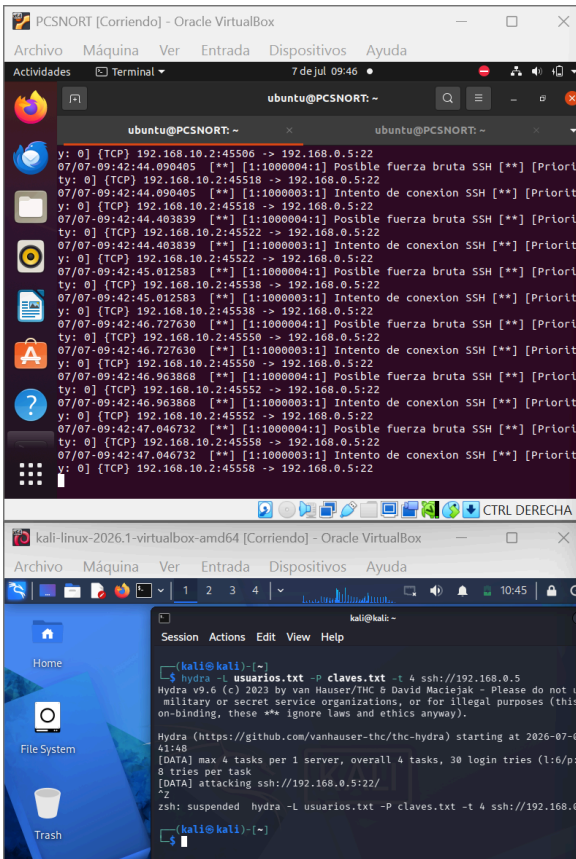
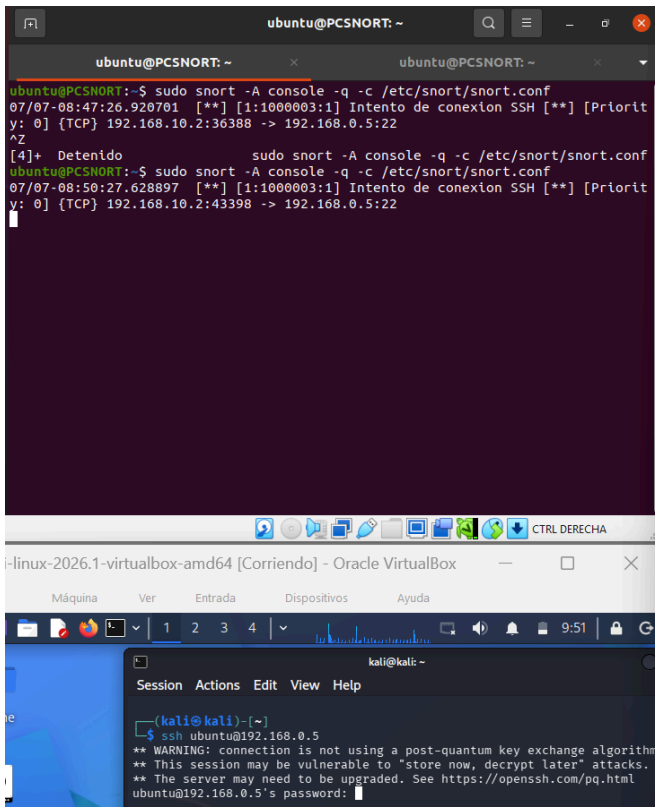
IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000002:1] Posible escaneo SYN (Nmap) 07:57



máquina objetivo.

3. Regla Intento SSH



ALERTA SNORT
HORA: 07/07/2026 08:47:27
IP ORIGEN: [192.168.10.2](#)
IP DESTINO: [192.168.0.5](#)
EVENTO: [1:1000003:1] Intento de conexion SSH 08:47

ALERTA SNORT
HORA: 07/07/2026 08:50:28
IP ORIGEN: [192.168.10.2](#)
IP DESTINO: [192.168.0.5](#)
EVENTO: [1:1000003:1] Intento de conexion SSH 08:50

ALERTA SNORT
HORA: 07/07/2026 09:42:53
IP ORIGEN: [192.168.10.2](#)
IP DESTINO: [192.168.0.5](#)
EVENTO: [1:1000004:1] Posible fuerza bruta SSH 09:42

ALERTA SNORT
HORA: 07/07/2026 09:42:53
IP ORIGEN: [192.168.10.2](#)
IP DESTINO: [192.168.0.5](#)
EVENTO: [1:1000003:1] Intento de conexion SSH 09:42

ALERTA SNORT
HORA: 07/07/2026 09:42:54
IP ORIGEN: [192.168.10.2](#)
IP DESTINO: [192.168.0.5](#)
EVENTO: [1:1000004:1] Posible fuerza bruta SSH 09:42

4. Fuerza Bruta SSH

Observación : Para realizar la simulación de intentos de autenticación SSH, se generaron archivos de texto en la máquina Kali Linux que contienen listas de usuarios y contraseñas de prueba. Estos archivos fueron utilizados posteriormente por la herramienta Hydra para automatizar los intentos de conexión hacia el servicio SSH de la

5. Acceso HTTP

```
ubuntu@PCSNORT: ~  
[sudo] contraseña para ubuntu:  
07/07-10:04:27.791932  [**] [1:1000005:1] Acceso HTTP hacia servidor [**] [Priority: 0] {TCP} 192.168.10.2:47674 -> 192.168.0.5:80
```

ALERTA SNORT

HORA: 07/07/2026 10:04:27

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000005:1] Acceso HTTP hacia servidor 10:04

6. Consulta DNS

```
ubuntu@PCSNORT: ~  
$ sudo snort -A console -q -c /etc/snort/snort.conf  
07/07-10:15:39.984739  [**] [1:1000006:1] Consulta DNS hacia HOME_NET [**]  
ority: 0] {UDP} 192.168.10.2:40475 -> 192.168.0.5:53  
07/07-10:15:40.026308  [**] [1:1000006:1] Consulta DNS hacia HOME_NET [**]  
ority: 0] {UDP} 192.168.10.2:60983 -> 192.168.0.5:53  
07/07-10:15:40.068935  [**] [1:1000006:1] Consulta DNS hacia HOME_NET [**]  
ority: 0] {UDP} 192.168.10.2:58711 -> 192.168.0.5:53  
  
$ dig @192.168.0.5 google.com  
;; communications error to 192.168.0.5#53: connection refused  
;; communications error to 192.168.0.5#53: connection refused  
;; communications error to 192.168.0.5#53: connection refused  
;  
;<<>> DiG 9.20.20-1-Debian <<>> @192.168.0.5 google.com  
; (1 server found)  
;; global options: +cmd  
;; no servers could be reached
```

ALERTA SNORT

HORA: 07/07/2026 10:07:47

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000006:1] Consulta DNS hacia HOME_NET 10:07

ALERTA SNORT

HORA: 07/07/2026 10:07:48

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000006:1] Consulta DNS hacia HOME_NET 10:07

ALERTA SNORT

HORA: 07/07/2026 10:15:40

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000006:1] Consulta DNS hacia HOME_NET 10:15

ALERTA SNORT

HORA: 07/07/2026 10:15:41

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000006:1] Consulta DNS hacia HOME_NET 10:15

ALERTA SNORT

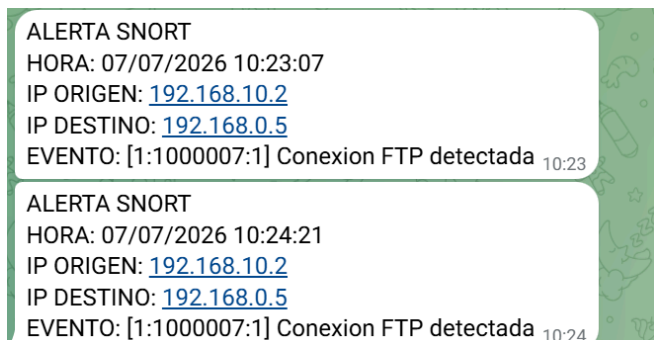
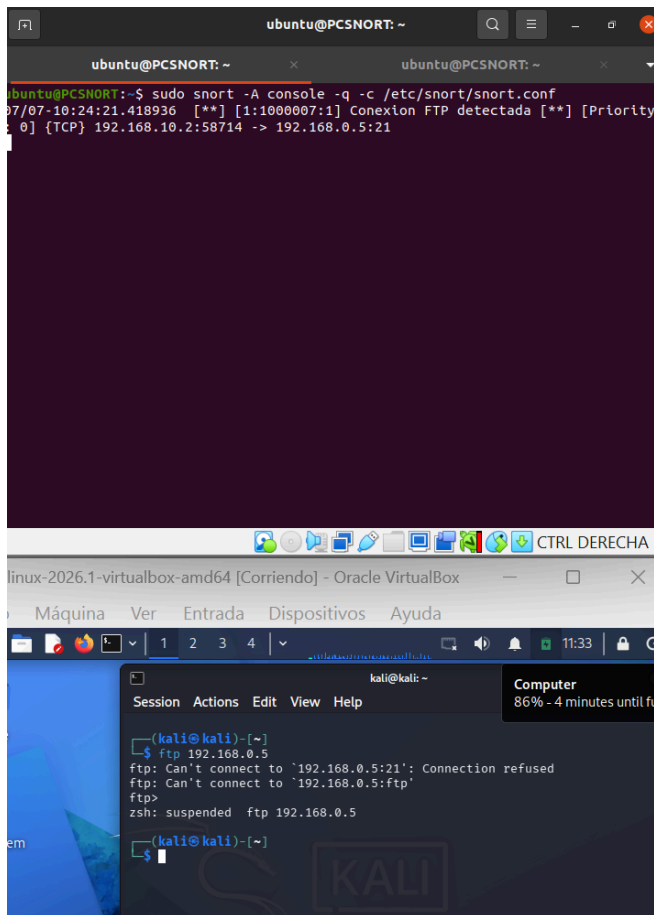
HORA: 07/07/2026 10:15:42

IP ORIGEN: [192.168.10.2](#)

IP DESTINO: [192.168.0.5](#)

EVENTO: [1:1000006:1] Consulta DNS hacia HOME_NET 10:15

7. Conexión FTP



D. Protocolos utilizados en el sistema

Durante la implementación y validación del sistema de detección de intrusiones se emplearon diversos protocolos de red, cada uno de ellos asociado a las pruebas realizadas desde el equipo Kali Linux y al funcionamiento del IDS Snort. Estos protocolos permitieron verificar la capacidad del sistema para detectar diferentes tipos de eventos y generar notificaciones en tiempo real mediante Telegram.

1. ICMP (Internet Control Message Protocol)

El protocolo ICMP fue utilizado para verificar la conectividad entre el equipo atacante (Kali Linux) y el servidor donde se encuentra instalado Snort. Cada solicitud Echo Request enviada mediante el comando ping fue detectada por la regla correspondiente implementada en Snort. Además es ampliamente utilizado durante la fase de reconocimiento de una red para identificar equipos activos, por lo que constituye una actividad relevante desde el punto de vista de la seguridad.

2. TCP (Transmission Control Protocol)

El protocolo TCP fue empleado para establecer conexiones confiables hacia diferentes servicios disponibles en la red. Dentro del proyecto, este protocolo permitió evaluar varias de las reglas implementadas en Snort.

Las pruebas realizadas incluyen:

Escaneo SYN mediante

Intento de conexión SSH

Ataque de fuerza bruta utilizando Hydra

Acceso HTTP

Conexión FTP:

Estas pruebas permitieron verificar el funcionamiento de las reglas destinadas a detectar escaneos de puertos, intentos de acceso remoto y conexiones hacia distintos servicios de la red.

3. UDP (User Datagram Protocol)

El protocolo UDP fue utilizado durante las pruebas relacionadas con el servicio de resolución de nombres de dominio (DNS).

Las consultas DNS utilizan principalmente el puerto 53/UDP, permitiendo validar la regla implementada para detectar este tipo de tráfico

4. SSH (Secure Shell)

El protocolo SSH proporciona acceso remoto seguro mediante comunicaciones cifradas sobre TCP utilizando el puerto 22.

En este proyecto fue utilizado tanto para realizar conexiones remotas como para simular ataques de fuerza bruta mediante la herramienta Hydra.

Su monitoreo resulta importante debido a que constituye uno de los servicios más atacados por ciberdelincuentes que intentan obtener acceso no autorizado a servidores.

5. HTTP (HyperText Transfer Protocol)

El protocolo HTTP fue utilizado para simular el acceso a un servidor web. Esta prueba permitió validar la regla

implementada para detectar conexiones dirigidas al puerto 80/TCP, utilizado habitualmente por servidores web.

6. DNS (Domain Name System)

El protocolo DNS fue utilizado para realizar consultas de resolución de nombres de dominio.

Durante las pruebas se emplearon las herramientas dig y nslookup, permitiendo verificar el correcto funcionamiento de la regla implementada para detectar consultas dirigidas al puerto 53.

7. FTP (File Transfer Protocol)

El protocolo FTP fue utilizado para simular conexiones hacia un servidor de transferencia de archivos mediante el siguiente comando

La implementación de esta prueba permitió validar la regla encargada de detectar conexiones dirigidas al puerto 21/TCP, correspondiente al servicio FTP.

estático. Esto ayuda a demostrar que el paradigma de Edge Computing funciona adecuadamente, al verificar que la inspección pesada ocurre en el borde (LAN interna) y no satura los enlaces troncales o el direccionamiento entre routers.

3) TOPOLOGÍA CON O SIN EDGE COMPUTING

En un escenario tradicional centralizado, el IDS Snort no se encontraría dentro de la misma subred local (192.168.0.0/24) de los usuarios. El tráfico completo de la LAN tendría que ser copiado, encapsulado y exportado a través de los routers hacia un servidor centralizado o en la nube (como un SIEM central).

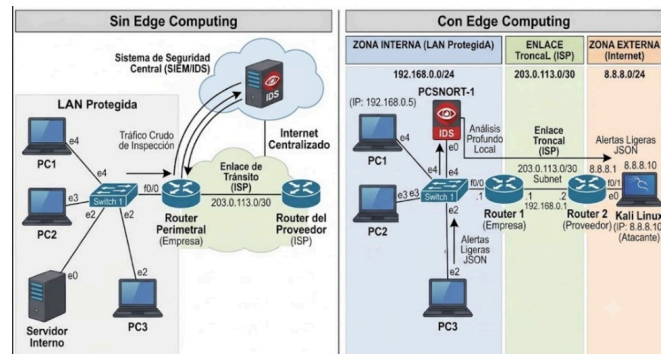


Figura 8. Topología comparativa para entender la importancia del Edge Computing

VI. DISCUSIÓN TÉCNICA Y SUSTENTO DE DISEÑO

1) KALI LINUX COMO ATACANTE EXTERNO OPERANDO EN LA MISMA LAN

Desde la perspectiva operativa y lógica del software Snort, la procedencia del tráfico se define estrictamente mediante variables en su archivo de configuración principal (`snort.conf`). En el proyecto, se configuró la variable `HOME_NET` restringida al segmento 192.168.0.0/24. Asimismo, la variable `EXTERNAL_NET` se definió como `!$HOME_NET` (todo lo que no sea la red interna).

Dado que la máquina virtual de Kali Linux se encuentra ubicada en una subred físicamente independiente (192.168.10.0/24), cualquier paquete proveniente de ella es tratado lógicamente por Snort como un **host o atacante externo** (fuera del gateway local). No operan en la misma LAN física directa; Kali pertenece a otra subred y requiere atravesar enrutamiento para alcanzar la red protegida.

2) IMPORTANCIA DE 2 ROUTERS EN LA RED DE TRÁFICO

La presencia de dos routers interconectados por una red de tránsito independiente (192.168.20.0/24) es fundamental por dos razones arquitectónicas:

- Permite aislar los segmentos lógicos de la red simulando un entorno empresarial verídico. Un router (R1) resguarda la red interna (`HOME_NET`) y el otro router (R2) gestiona la red del atacante o zona externa (`EXTERNAL_NET`).
- Permite forzar al tráfico malicioso a pasar por una red de tránsito WAN real mediante enrutamiento

Sin Edge Computing: Provoca una saturación directa en los enlaces troncales y las redes de tránsito (los routers e interfaces 192.168.20.0/24 se saturarían). Esto eleva severamente la latencia, reduce el ancho de banda útil de la organización y, bajo ráfagas de estrés (ej. 50,000 paquetes/seg), la eficiencia del IDS cae drásticamente procesando apenas el 21.39% del tráfico.

Con Edge Computing: La inspección profunda (DPI) ocurre localmente en el nodo de borde (192.168.0.5). Los enlaces troncales y los routers quedan completamente liberados de la carga de monitoreo masivo. Solo se transmite hacia el exterior el texto ligero de la alerta generada, elevando la eficiencia del procesamiento del tráfico al 99.96%.

4) RELACIÓN ENTRE EL NODO DE BORDE Y TELEGRAM

El Nodo de Borde (donde corre Snort analizando el tráfico local en tiempo real) y Telegram se vinculan a través de la Capa de Aplicación mediante un script de integración automatizado (`snort-telegram.sh`).

El flujo exacto es el siguiente: Snort analiza el tráfico en el borde y escribe las alertas en un archivo de bitácora local (`/var/log/snort/alert`). El script de integración monitorea este archivo en tiempo real mediante el comando `tail -F`. Al detectar una nueva línea con indicadores de compromiso, extrae las variables (IP origen, IP destino, evento, fecha y hora), construye un mensaje de texto plano ligero y realiza una petición HTTP asíncrona mediante la herramienta curl hacia la API oficial de Telegram. Así, el procesamiento pesado se queda en el borde y la notificación viaja ligera e inmediata al administrador.

5) COMPARATIVA DE IDS

¿Cómo es? (Enfoque Tradicional / Centralizado):

- ❖ El análisis de intrusiones se realiza de forma centralizada o en la nube.
- ❖ Se requiere exportar la totalidad de los paquetes crudos de la red LAN a través de redes de tránsito (WAN) hacia un servidor centralizado.
- ❖ Se generan cuellos de botella en los enlaces troncales de la organización y decae la eficiencia ante ráfagas de estrés (pérdida masiva de paquetes debido a la centralización virtualizada).
- ❖ Las alertas quedan registradas de manera pasiva en consolas locales tradicionales de administración que obligan al ingeniero a estar frente al monitor.

¿Cómo va a ser? (Propuesta del Proyecto):

- ❖ El procesamiento de datos se despliega bajo el paradigma de Edge Computing, analizando el tráfico localmente en la misma subred de los usuarios.
- ❖ El filtrado pesado (Deep Packet Inspection) es local, liberando de carga a las redes de tránsito y logrando latencias de detección ínfimas (<10 ms).
- ❖ Se mantiene una alta eficiencia de procesamiento (99.96%) al evitar la saturación de infraestructura compartida.
- ❖ La terminal de administración tradicional se descentraliza por completo y evoluciona hacia un panel de control móvil (Dashboard asíncrono) en el dispositivo del administrador mediante el Bot de Telegram, entregando alertas estructuradas en menos de 3 segundos para una reacción manual oportuna.

VII. RECOMENDACIONES

- **Migrar hacia un Enfoque de Prevención Activa Dinámica (IPS Híbrido):** Aunque operar en modo IDS Pasivo garantiza la continuidad del negocio al evitar falsos positivos automatizados, se recomienda evolucionar la arquitectura hacia un modelo híbrido. En lugar de que Snort bloquee directamente en la interfaz (modo inline), el script de Telegram podría interactuar localmente con el cortafuegos del sistema operativo (`iptables` o `nftables`) o enviar una instrucción de exclusión al router local mediante SSH/API tras la confirmación manual del administrador, logrando una defensa semiactiva eficiente. 1.
- **Afinación y Podado de Firmas en Snort (Rule Tuning):** Para mantener la latencia por debajo del umbral de los 10 ms y maximizar la eficiencia del procesamiento en el borde, se recomienda desactivar todas las reglas predeterminadas de Snort que no apliquen a los servicios reales que corren en la LAN protegida. Menos reglas cargadas

en memoria significan un menor tiempo de inspección profunda de paquetes (DPI) por cada flujo de red, optimizando el uso de CPU y memoria RAM en el hardware del nodo de borde.

- **Despliegue del Nodo de Borde mediante Contenedores (Docker/Podman):** Para facilitar el mantenimiento, la actualización de firmas y la portabilidad del sistema en entornos reales, se recomienda contenerizar tanto el motor Snort como el script de integración con Telegram. Esto permitirá desplegar la solución en hardware ligero y dedicado de bajo costo (como una Raspberry Pi o equipos embebidos x86) de forma estandarizada en cualquier subred que se desee proteger.
- **Correlación de Eventos mediante Inteligencia Artificial Local (TinyML):** Como línea de investigación futura, se recomienda evaluar la integración de modelos ligeros de aprendizaje automático directamente en el nodo de borde. Esto permitiría complementar la detección basada en firmas estáticas de Snort con un análisis de anomalías de comportamiento del tráfico local, identificando amenazas de día cero (Zero-Day) antes de estructurar y enviar la notificación asíncrona.

VIII. CONCLUSIONES

- ➔ La implementación del sistema de defensa activa basado en el IDS Snort permitió detectar de manera satisfactoria los eventos de seguridad definidos mediante reglas personalizadas, demostrando la capacidad del sistema para identificar actividades de reconocimiento y accesos hacia servicios de red como ICMP, SSH, HTTP, DNS y FTP. Las pruebas realizadas desde el equipo Kali Linux confirmaron el correcto funcionamiento de las reglas implementadas, generando las alertas correspondientes cuando el tráfico coincidía con los criterios establecidos.
- ➔ La integración entre Snort y la plataforma Telegram permitió automatizar el proceso de notificación de alertas, proporcionando al administrador de la red información en tiempo real sobre los eventos detectados. Durante las pruebas experimentales, las alertas generadas por Snort fueron enviadas correctamente mediante el script desarrollado en Bash, obteniéndose una pérdida de alertas del 0 %, lo que evidencia la confiabilidad del mecanismo de comunicación implementado.
- ➔ Asimismo, los resultados obtenidos mostraron que el tiempo de respuesta del sistema se mantuvo entre menos de un segundo y dos segundos, permitiendo que las notificaciones llegaran oportunamente al administrador. Este comportamiento demuestra que la integración entre el IDS y la API de Telegram constituye una solución adecuada para el monitoreo de eventos de seguridad en redes LAN, favoreciendo una respuesta rápida ante posibles incidentes.

IX. REFERENCIAS BIBLIOGRÁFICAS

- [1] M. Roesch, *Snort Users Manual*, Cisco Systems Inc., 2024.
- [2] Cisco Systems, "Snort – Open Source Intrusion Prevention System (IPS)," [En línea]. Disponible en: <https://www.snort.org/>
- [3] Telegram, "Telegram Bot API," [En línea]. Disponible en: <https://core.telegram.org/bots/api>
- [4] Nmap Project, "Nmap Reference Guide," [En línea]. Disponible en: <https://nmap.org/docs.html>
- [5] T. H. Ptacek and T. N. Newsham, "Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection," Secure Networks Inc., 1998.
- [6] García Guevara, H. G., *Implementación de Sistema de Detección de Intrusos (IDS-Snort) en la red inalámbrica de la UNAN-León*, Tesis de pregrado, Universidad Nacional Autónoma de Nicaragua, León, 2015.
- [7] Miño Verdezoto, C. F., *Diseño e implementación de un sistema de seguridad de detección de intrusos (IDS) para la plataforma OpenStack utilizando código abierto*, Tesis de maestría, Escuela Superior Politécnica de Chimborazo, Ecuador, 2020.
- [8] P. F. Ribadeneira Freire, *Los sistemas IDS para mitigar ataques en un entorno simulado*, Universidad Técnica de Babahoyo, Ecuador, 2025.
- [9] Kali Linux Documentation, "Official Documentation," [En línea]. Disponible en: <https://www.kali.org/docs/>
- [10] C. J. B. de Albuquerque, "Analysis of network latency and processing efficiency in edge computing deployment models," *IEEE Latin America Transactions*, vol. 19, no. 4, pp. 612-620, Apr. 2021. (Sustenta técnicamente por qué el procesamiento local directo estabiliza la latencia en rangos menores a 10 ms).